유한체 연산

개요

유한체(Finite Field)는 zkSNARK 프로토콜의 모든 연산이 수행되는 수학적 구조입니다. 효율적인 다항식 연산과 FFT(Fast Fourier Transform) 구현의 기초가 됩니다.

유한체의 정의

기본 개념

- **유한체 F_q**: q개의 원소를 가진 체
- **소수 체 F_p**: p가 소수일 때의 유한체
- **확장 체 F_{p^k}**: F_p의 k차 확장체

체의 공리

1. **덧셈에 대한 아벨군**: (F, +)
2. **곱셈에 대한 아벨군**: (F{0}, ×)
3. **분배법칙**: a(b + c) = ab + ac

기본 연산

덧셈과 뺄셈

```
template<typename FieldT>
```

```
FieldT field_add(const FieldT&  a, const FieldT&  b) {
return  FieldT((a.as_bigint() + b.as_bigint()) % FieldT::mod);
}

template<typename FieldT>
FieldT field_sub(const FieldT&  a, const FieldT&  b) {
return  FieldT((a.as_bigint() - b.as_bigint() + FieldT::mod) % FieldT::mod);
}
```

곱셈

```
template<typename FieldT>
FieldT field_mul(const FieldT&  a, const FieldT&  b) {
return  FieldT((a.as_bigint() * b.as_bigint()) % FieldT::mod);
}
```

역원 계산

```
template<typename FieldT>
FieldT field_inv(const FieldT&  a) {
// 확장 유클리드 알고리즘 사용
return  a.inverse();
```

```
}
```

FFT와 다항식 연산

이산 푸리에 변환의 원리

- **단위근**: ω^n = 1을 만족하는 ω
- **원시 단위근**: ω^k ≠ 1 (1 ≤ k < n)을 만족하는 ω

FFT 구현

```
template<typename FieldT>
void  fft(vector<FieldT>&  a, bool  invert = false) {
size_t n = a.size();
// 비트 역순 정렬
for (size_t i = 1, j = 0; i < n; i++) {
size_t bit = n >> 1;
for (; j & bit; bit >>= 1) {
j ^= bit;
}
j ^= bit;
if (i < j) {
swap(a[i], a[j]);
}
}
```

```
// FFT 계산
for (size_t len = 2; len <= n; len <<= 1) {
FieldT wlen = invert ?
FieldT::multiplicative_generator().inverse().pow((FieldT::mod - 1) / len) :
FieldT::multiplicative_generator().pow((FieldT::mod - 1) / len);
for (size_t i = 0; i < n; i += len) {
FieldT w = FieldT::one();
for (size_t j = 0; j < len / 2; j++) {
FieldT u = a[i + j];
FieldT v = a[i + j + len / 2] * w;
a[i + j] = u + v;
a[i + j + len / 2] = u - v;
w *= wlen;
}
}
}
if (invert) {
FieldT n_inv = FieldT(n).inverse();
for (FieldT& x : a) {
x *= n_inv;
}
}
```

```
}
```

다항식 곱셈

```
template<typename FieldT>
vector<FieldT> polynomial_multiply(
const vector<FieldT>&  a,
const vector<FieldT>&  b
) {
vector<FieldT> fa(a.begin(), a.end());
vector<FieldT> fb(b.begin(), b.end());
size_t n = 1;
while (n < a.size() + b.size()) {
n <<= 1;
}
fa.resize(n);
fb.resize(n);
fft(fa, false);
fft(fb, false);
for (size_t i = 0; i < n; i++) {
fa[i] *= fb[i];
}
fft(fa, true);
```

```
vector<FieldT> result(a.size() + b.size() - 1);

for (size_t i = 0; i < result.size(); i++) {

result[i] = fa[i];

}

return result;

}
```

NTT (Number Theoretic Transform)

NTT의 장점

- **정확한 정수 연산**: 부동소수점 오차 없음
- **효율적인 구현**: 유한체 연산의 최적화
- **큰 수 처리**: 임의 정밀도 정수 연산

NTT 적합한 소수

```
// 2^k * q + 1 형태의 소수 (k ≥ 20)

const  uint64_t NTT_MOD = 998244353; // 119 * 2^23 + 1

const  uint64_t NTT_ROOT = 3; // 원시근
```

NTT 구현 최적화

```
template<typename FieldT>

class OptimizedNTT {
```

```
private:
vector<FieldT> roots;
vector<FieldT> inv_roots;
public:
OptimizedNTT(size_t max_size) {
// 미리 계산된 단위근 테이블
precompute_roots(max_size);
}
void ntt(vector<FieldT>& a, bool invert = false) {
size_t n = a.size();
// 비트 역순 정렬 (최적화된 버전)
bit_reverse(a);
// Cooley-Tukey FFT
for (size_t len = 2; len <= n; len <<= 1) {
for (size_t i = 0; i < n; i += len) {
for (size_t j = 0; j < len / 2; j++) {
FieldT w = invert ?
inv_roots[len / 2 + j] :
roots[len / 2 + j];
FieldT u = a[i + j];
FieldT v = a[i + j + len / 2] * w;
a[i + j] = u + v;
a[i + j + len / 2] = u - v;
```

```
}
}
}
if (invert) {
FieldT n_inv = FieldT(n).inverse();
for (FieldT& x : a) {
x *= n_inv;
}
}
}
};
```

이진 확장 체와 소수 체

이진 확장 체 F_{2^k}

- **표현**: 다항식 기저 또는 정규 기저
- **연산**: XOR 기반 덧셈, 다항식 곱셈
- **장점**: 하드웨어 구현에 효율적

```
class BinaryField {
private:
uint64_t value;
static const uint64_t irreducible_poly = 0x1B; // x^8 + x^4 + x^3 + x + 1
```

```
public:
BinaryField operator+(const BinaryField&  other) const {
return  BinaryField(value ^ other.value);
}
BinaryField operator*(const BinaryField&  other) const {
uint64_t result = 0;
uint64_t a = value;
uint64_t b = other.value;
while (b) {
if (b & 1) {
result ^= a;
}
a <<= 1;
if (a & 0x100) {
a ^= irreducible_poly;
}
b >>= 1;
}
return  BinaryField(result);
}
};
```

소수 체 F_p

- **표현**: 정수 모듈로 p
- **연산**: 모듈러 산술
- **장점**: 수학적 분석이 용이

구현 고려사항

성능 최적화

1. **Montgomery 곱셈**: 모듈러 곱셈 최적화
2. **배치 처리**: SIMD 명령어 활용
3. **메모리 지역성**: 캐시 효율적인 접근 패턴

보안 고려사항

1. **상수 시간 구현**: 사이드 채널 공격 방지
2. **안전한 난수 생성**: 암호학적으로 안전한 무작위성
3. **메모리 보안**: 민감한 데이터의 안전한 삭제

다음 단계

다음 문서에서는 다항식 이론을 통해 유한체에서의 다항식 연산을 더 자세히 살펴보겠습니다.

← 수학적 기초 목차 | 다음: 다항식 이론 →